

Codroid SDK

1.0.0

制作者 Doxygen 1.9.1

1 Codroid SDK	1
1.1 * 特点	1
1.2 [Dir] 项目结构	1
1.3 [Build] 构建指南	2
1.3.1 1. Linux (Ubuntu) 编译	2
1.3.2 2. Windows (Visual Studio / MSVC) 编译	2
1.4 [PC] 如何在您的项目中使用	2
1.5 [Fast] 快速上手示例	3
1.6 [Warn] 注意事项	3
2 继承关系索引	5
2.1 类继承关系	5
3 类索引	7
3.1 类列表	7
4 文件索引	9
4.1 文件列表	9
5 类说明	11
5.1 Codroid::CodroidControllInterface 类参考	11
5.1.1 成员函数说明	14
5.1.1.1 calculateRelativePose()	14
5.1.1.2 clearError()	14
5.1.1.3 clearStartLine()	16
5.1.1.4 connect()	16
5.1.1.5 enterRemoteScriptMode()	17
5.1.1.6 forwardKinematics()	17
5.1.1.7 getAI()	17
5.1.1.8 getAO()	18
5.1.1.9 getDI()	18
5.1.1.10 getDO()	18
5.1.1.11 getGlobalVars()	20
5.1.1.12 getIOValues()	20
5.1.1.13 getProjectVar()	21
5.1.1.14 getRegisterValue()	21
5.1.1.15 getRegisterValues()	21
5.1.1.16 inverseKinematics()	22
5.1.1.17 jog()	22
5.1.1.18 jogHeartbeat()	22
5.1.1.19 movC()	23
5.1.1.20 movCircle()	23
5.1.1.21 move()	24
5.1.1.22 moveTo()	24

5.1.1.23 moveToHeartbeat()	24
5.1.1.24 movJ() [1/2]	25
5.1.1.25 movJ() [2/2]	25
5.1.1.26 movL() [1/2]	26
5.1.1.27 movL() [2/2]	26
5.1.1.28 pauseMove()	26
5.1.1.29 pauseProject()	27
5.1.1.30 printResponse()	27
5.1.1.31 removeExtendArray()	27
5.1.1.32 removeGlobalVars()	28
5.1.1.33 resumeMove()	28
5.1.1.34 resumeProject()	29
5.1.1.35 RS485flush()	29
5.1.1.36 RS485init()	29
5.1.1.37 RS485read()	30
5.1.1.38 RS485write()	30
5.1.1.39 runProject()	31
5.1.1.40 runProjectByIndex()	31
5.1.1.41 runScript()	31
5.1.1.42 runStep()	32
5.1.1.43 saveGlobalVars()	32
5.1.1.44 sendCommand()	33
5.1.1.45 setAO()	33
5.1.1.46 setAutoSpeedRate()	33
5.1.1.47 setCollisionSensitivity()	34
5.1.1.48 setDO()	34
5.1.1.49 setExtendArrayType()	35
5.1.1.50 setIOValue()	35
5.1.1.51 setManualSpeedRate()	36
5.1.1.52 setPayload()	36
5.1.1.53 setRegisterValue()	36
5.1.1.54 setStartLine()	37
5.1.1.55 startControl()	37
5.1.1.56 startDataPush()	38
5.1.1.57 startDrag()	38
5.1.1.58 stopControl()	39
5.1.1.59 stopDataPush()	39
5.1.1.60 stopDrag()	39
5.1.1.61 stopJog()	40
5.1.1.62 stopMove()	40
5.1.1.63 stopProject()	40
5.1.1.64 switchOff()	41
5.1.1.65 switchOn()	41

5.1.1.66 toActual()	41
5.1.1.67 toAuto()	42
5.1.1.68 toManual()	42
5.1.1.69 toRemote()	43
5.1.1.70 toSimulation()	43
5.2 Codroid::CodroidException 类参考	43
5.2.1 详细描述	44
5.3 Codroid::CodroidSubscriber 类参考	44
5.3.1 详细描述	44
5.3.2 成员函数说明	44
5.3.2.1 connect()	44
5.3.2.2 setLogCallback()	45
5.3.2.3 setPostureCallback()	45
5.3.2.4 setStatusCallback()	45
5.3.2.5 setVarCallback()	46
5.3.2.6 subscribe()	46
5.4 Codroid::FKParams 结构体参考	46
5.4.1 详细描述	47
5.5 Codroid::IKParams 结构体参考	47
5.5.1 详细描述	47
5.6 Codroid::IOInfo 结构体参考	48
5.6.1 详细描述	48
5.7 Codroid::JogParams 结构体参考	48
5.7.1 详细描述	49
5.8 Codroid::MoveInstruction 结构体参考	49
5.8.1 详细描述	49
5.9 Codroid::MoveToParams 结构体参考	50
5.9.1 详细描述	50
5.10 Codroid::MoveToTarget 结构体参考	50
5.10.1 详细描述	51
5.11 Codroid::ProjectState 结构体参考	51
5.11.1 详细描述	51
5.12 Codroid::RegisterInfo 结构体参考	51
5.12.1 详细描述	52
5.13 Codroid::RelativePoseParams 结构体参考	52
5.13.1 详细描述	52
5.14 Codroid::Response 结构体参考	53
5.14.1 详细描述	53
5.15 Codroid::RobotPosture 结构体参考	53
5.15.1 详细描述	53
5.16 Codroid::RobotStatus 结构体参考	54
5.16.1 详细描述	54
5.17 Codroid::Variable 结构体参考	54

5.17.1 详细描述	55
6 文件说明	57
6.1 include/Codroid/CodroidControllInterface.h 文件参考	57
6.1.1 详细描述	57
6.2 include/Codroid/CodroidDefine.h 文件参考	57
6.2.1 详细描述	59
6.2.2 函数说明	59
6.2.2.1 NLOHMANN_JSON_SERIALIZE_ENUM() [1/2]	59
6.2.2.2 NLOHMANN_JSON_SERIALIZE_ENUM() [2/2]	59
6.3 include/Codroid/CodroidSubscriber.h 文件参考	60
6.3.1 详细描述	60

Chapter 1

Codroid SDK

Codroid SDK 是一个用于机械臂远程控制的跨平台 C++ 开发工具包。它通过 TCP 通信协议实现与机器人的交互，涵盖了运动控制 (movJ/movL/movC)、IO 管理、寄存器操作、数据流订阅及实时控制等核心功能。

1.1 * 特点

- **** 开箱即用 ****: 底层依赖库 (Asio 和 nlohmann/json) 已内置于 third_party 文件夹中, 克隆仓库后无需额外安装系统依赖即可直接编译。
- **** 双通道架构 ****: 支持独立的指令通道与状态订阅通道, 确保高频运动指令下发与实时状态反馈互不干扰。
- **** 单位标准化 ****: SDK 内部已自动完成单位换算。
 - **** 长度单位 ****: 毫米 (mm)
 - **** 角度单位 ****: 角度 (deg) (机器人返回的弧度数据已在内部自动转换为角度, 并保留 3 位小数)。
- **** 跨平台兼容 ****: 完美支持 Linux (GCC) 与 Windows (Visual Studio / MSVC) 编译环境。

1.2 [Dir] 项目结构

- include/Codroid/ - SDK 对外公开接口头文件。
- src/ - SDK 核心逻辑实现源代码。
- third_party/ - 内置依赖库 (Asio Standalone, nlohmann/json)。
- examples/ - 功能测试示例 (点动、路径运动、IO 测试、寄存器测试等)。
- build_linux.sh - Linux 一键构建脚本。
- build_msvc.bat - Windows MSVC 一键构建脚本。

1.3 [Build] 构建指南

1.3.1 1. Linux (Ubuntu) 编译

在项目根目录下执行以下命令：

```
chmod +x build_linux.sh
./build_linux.sh
```

构建产物：

- 动态库：build_linux/libCodroid.so
- 示例程序：位于 build_linux/ 目录下的各个可执行文件（如 move_test）。

运行示例：

```
cd build_linux
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
./move_test
```

1.3.2 2. Windows (Visual Studio / MSVC) 编译

在 Windows 环境下，确保已安装 CMake 和 Visual Studio 2019/2022，双击运行：

```
build_msvc.bat
```

构建产物：

- build_msvc/Debug
- build_msvc/Release。

关键文件：Codroid.dll（运行时必须），Codroid.lib（编译链接必须）。

1.4 [PC] 如何在您的项目中使用

由于本 SDK 未采用 PIMPL 模式隐藏底层依赖，在您的第三方 C++ 项目中引入本 SDK 时，请务必进行以下配置：

1. 包含目录 (Include Directories)

将以下两个路径添加到您的项目包含路径（Include Path）中：

- CodroidSDK/include
- CodroidSDK/third_party（由于头文件嵌套，必须包含此路径，否则找不到 asio.hpp）

2. 预处理器定义 (Preprocessor Definitions)

必须添加以下宏定义：

- ASIO_STANDALONE（告诉 Asio 使用标准版而非 Boost 版）

3. Windows 环境额外配置

- 链接库：将 Codroid.lib 添加到链接器的附加依赖项。
- 字符编码：建议开启 /utf-8 编译选项，以确保中文注释不会引起编译语法错误。
- 运行时：程序运行时必须将 Codroid.dll 放置在与 .exe 文件相同的目录下。

1.5 [Fast] 快速上手示例

```
{c++}
#include <iostream>
#include <vector>
#include "Codroid/CodroidControlInterface.h"
int main() {
    Codroid::CodroidControlInterface robot;
    // 1. 连接机器人 (默认端口 9001)
    if (robot.connect("192.168.1.136", 9001)) {
        std::cout << "Connected to Codroid Robot!" << std::endl;
        // 2. 获取当前笛卡尔位姿 (单位: mm, deg)
        // 内部已自动同步并跳过初始零位包
        std::vector<double> currentPos = robot.getCartesianPosition();
        std::cout << "Current X: " << currentPos[0] << " mm" << std::endl;
        // 3. 关节运动
        // 目标: 关节角度 [j1..j6]
        std::vector<double> home = {0.0, 0.0, 90.0, 0.0, 90.0, 0.0};
        robot.movJ(home, 50.0, 100.0);
        // 4. 断开连接
        robot.disconnect();
    } else {
        std::cerr << "Connection failed!" << std::endl;
    }
    return 0;
}
```

1.6 [Warn] 注意事项

1. 状态同步延迟: 受限于机器人端状态推送固定为 1 秒/次, Wait 系列阻塞函数在判断机器人停止时可能会存在最高 1 秒的自然同步延迟。
2. 空数组崩溃预防: SDK 内部实现了防御性编程, 发送指令时会自动剔除空的 coor 或 tool 数组字段, 以规避机器人后端处理空数组时的崩溃 Bug。
3. 多线程安全: sendCommand 内部已通过 std::mutex 加锁。支持在多线程环境下并发调用非阻塞运动指令, 但建议在同一时刻只由一个线程控制机器人运动。
4. 单位验证: 请注意, 机器人推送的 joint 数据在 SDK 内部被视为角度 (Degrees)。若您的机器人固件返回的是弧度, SDK 将按照 1rad = 57.295deg 进行自动转换。

Chapter 2

继承关系索引

2.1 类继承关系

此继承关系列表按字典顺序粗略的排序:

Codroid::CodroidControllInterface	11
Codroid::CodroidSubscriber	44
Codroid::FKParams	46
Codroid::IKParams	47
Codroid::IOInfo	48
Codroid::JogParams	48
Codroid::MoveInstruction	49
Codroid::MoveToParams	50
Codroid::MoveToTarget	50
Codroid::ProjectState	51
Codroid::RegisterInfo	51
Codroid::RelativePoseParams	52
Codroid::Response	53
Codroid::RobotPosture	53
Codroid::RobotStatus	54
std::runtime_error	
Codroid::CodroidException	43
Codroid::Variable	54

Chapter 3

类索引

3.1 类列表

这里列出了所有类、结构、联合以及接口定义等，并附带简要说明:

Codroid::CodroidControllInterface	11
Codroid::CodroidException	
Codroid SDK 专用异常类	43
Codroid::CodroidSubscriber	
用于通过独立 TCP 通道进行异步数据订阅的专用类。	44
Codroid::FKParams	
正解参数结构体	46
Codroid::IKParams	
逆解参数结构体	47
Codroid::IOInfo	
IO 信息结构体	48
Codroid::JogParams	
点动参数结构体	48
Codroid::MoveInstruction	
单条运动指令详情	49
Codroid::MoveToParams	
运动参数	50
Codroid::MoveToTarget	
运动目标位置	50
Codroid::ProjectState	
工程执行状态	51
Codroid::RegisterInfo	
寄存器信息结构体	51
Codroid::RelativePoseParams	
相对位姿计算参数	52
Codroid::Response	
SDK 标准响应结构体	53
Codroid::RobotPosture	
机器人实时位姿	53
Codroid::RobotStatus	
机器人运行状态	54
Codroid::Variable	
全局变量信息结构	54

Chapter 4

文件索引

4.1 文件列表

这里列出了所有文档化的文件，并附带简要说明:

include/Codroid/ CodroidControllInterface.h	
控制接口头文件	57
include/Codroid/ CodroidDefine.h	
定义文件	57
include/Codroid/CodroidExport.h	??
include/Codroid/ CodroidSubscriber.h	
CodroidSubscriber 类的头文件	60

Chapter 5

类说明

5.1 Codroid::CodroidControllInterface 类参考

Public 成员函数

- [CodroidControllInterface \(\)](#)
构造函数
- [~CodroidControllInterface \(\)](#)
析构函数
- [CodroidControllInterface \(const \[CodroidControllInterface\]\(#\) &\)=delete](#)
- [CodroidControllInterface & operator= \(const \[CodroidControllInterface\]\(#\) &\)=delete](#)
- [bool connect \(const std::string &ip, int port=9001\)](#)
连接机器人
- [void disconnect \(\)](#)
断开与机器人的连接
- [Response sendCommand \(const std::string &type, const json &data, int id\)](#)
向机器人发送指令
- [Response runScript \(const std::string &script, int id=1\)](#)
在机器人上运行脚本
- [Response enterRemoteScriptMode \(int id=1\)](#)
在机器人上进入远程脚本模式
- [Response runProject \(const std::string &projectid, int id=1\)](#)
在机器人上通过工程ID 运行工程
- [Response runProjectByIndex \(int index, int id=1\)](#)
在机器人上通过索引运行工程
- [Response runStep \(const std::string &projectid, int id=1\)](#)
在机器人上单步运行工程
- [Response pauseProject \(int id=1\)](#)
暂停机器人上正在运行的工程
- [Response resumeProject \(int id=1\)](#)
恢复机器人上暂停的工程
- [Response stopProject \(int id=1\)](#)
停止机器人上正在运行的工程
- [Response setStartLine \(int startline, int id=1\)](#)
设置机器人上工程执行的启动行
- [Response clearStartLine \(int id=1\)](#)

- 清除机器人上工程执行的启动行设置
- `Response getGlobalVars` (int id=1)
获取机器人上的全局变量列表
- `Response saveGlobalVars` (const std::map< std::string, `Variable` > &vars, int id=1)
将全局变量列表保存到机器人
- `Response removeGlobalVars` (const std::vector< std::string > &varNames, int id=1)
从机器人上删除全局变量
- `Response getProjectVar` (int id=1)
从机器人上获取工程变量的值
- `Response RS485init` (int baudrate, `RS485StopBits` stopBit=`RS485StopBits::One`, int dataBit=8, `RS485Parity` parity=`RS485Parity::None`, int id=1)
初始化机器人上的RS485 通信
- `Response RS485flush` (int id=1)
清空机器人上的RS485 通信缓冲区
- `Response RS485read` (int length, int timeout=5000, int id=1)
从机器人上的RS485 通信读取数据
- `Response RS485write` (const std::vector< uint8_t > &data, int id=1)
向机器人上的RS485 通信写入数据
- std::vector< double > `forwardKinematics` (const `FKParams` ¶ms, int id=1)
计算机人的正运动学
- std::vector< double > `inverseKinematics` (const `IKParams` ¶ms, int id=1)
计算机人的逆运动学
- std::vector< double > `calculateRelativePose` (const `RelativePoseParams` ¶ms, int id=1)
计算机人的相对姿态
- `Response jog` (const `JogParams` ¶ms, int id=1)
对机器人执行点动操作
- `Response stopJog` (int id=1)
停止机器人上的点动操作
- `Response jogHeartbeat` (int id=1)
发送点动操作的心跳信号, 至少每 500ms 发送一次
- `Response moveTo` (const `MoveToParams` ¶ms, int id=1)
将机器人移动到特定位置
- `Response moveToHeartbeat` (int id=1)
发送 moveTo 操作的心跳信号, 至少每 500ms 发送一次
- `Response setManualSpeedRate` (int speed, int id=1)
设置机器人的手动运动倍率
- `Response setAutoSpeedRate` (int speed, int id=1)
设置机器人的自动运动倍率
- `Response move` (const std::vector< `MoveInstruction` > &path, int id=1)
按指定路径移动机器人
- `Response movJ` (const `MoveInstruction` &inst, int id=1)
将机器人移动到特定关节位置
- `Response movJ` (const std::vector< double > &jp, double speed, double acc, int id=1)
将机器人移动到特定关节位置
- `Response movL` (const `MoveInstruction` &inst, int id=1)
按直线路径移动机器人
- `Response movL` (const std::vector< double > &cp, double speed, double acc, const std::vector< double > &coor={}, const std::vector< double > &tool={}, int id=1)
按直线路径移动机器人
- `Response movC` (const `MoveInstruction` &inst, int id=1)
按圆弧路径移动机器人

- [Response movCircle](#) (const [MoveInstruction](#) &inst, int id=1)
按圆弧路径移动机器人，形成一个完整的圆
- [Response pauseMove](#) (int id=1)
暂停机器人当前的运动
- [Response resumeMove](#) (int id=1)
恢复机器人暂停的运动
- [Response stopMove](#) (int id=1)
立即停止机器人当前的运动
- [Response switchOn](#) (int id=1)
上使能机器人
- [Response switchOff](#) (int id=1)
下使能机器人
- [Response toManual](#) (int id=1)
进入机器人的手动模式
- [Response toAuto](#) (int id=1)
进入机器人的自动模式
- [Response toRemote](#) (int id=1)
进入机器人的远程模式
- [Response toSimulation](#) (int id=1)
进入机器人的仿真模式
- [Response toActual](#) (int id=1)
进入机器人的实机模式
- [Response startDrag](#) (int id=1)
进入机器人的拖拽模式
- [Response stopDrag](#) (int id=1)
退出机器人的拖拽模式
- [Response clearError](#) (int id=1)
清除机器人上的错误
- `std::vector< IOInfo > getIOValues` (const `std::vector< IOInfo >` &queryList, int id=1)
获取机器人上IO 端口的状态
- `int getDI` (int port, int id=1)
获取数字输入端口的值
- `int getDO` (int port, int id=1)
获取数字输出端口的值
- `double getAI` (int port, int id=1)
获取模拟输入端口的值
- `double getAO` (int port, int id=1)
获取模拟输出端口的值
- [Response setIOValue](#) (const `std::string` &type, int port, double value, int id=1)
设置机器人上IO 端口的值
- [Response setDO](#) (int port, int value, int id=1)
设置数字输出端口的值
- [Response setAO](#) (int port, double value, int id=1)
设置模拟输出端口的值
- `std::vector< RegisterInfo > getRegisterValues` (const `std::vector< int >` &addresses, int id=1)
获取机器人上寄存器的值
- `double getRegisterValue` (const int address, int id=1)
获取机器人上单个寄存器的值
- [Response setRegisterValue](#) (int address, double value, int id=1)
设置机器人上寄存器的值
- [Response setExtendArrayType](#) (int index, `ExtendArrayType` type, int id=1)

- 设置机器人上扩展数组的数据类型
- `Response removeExtendArray` (int index, int id=1)
从机器人上删除一个扩展数组索引
- `Response startDataPush` (const std::string &ip, int port, int duration, int id=1)
开启从机器人到指定IP 和端口的数据推送流
- `Response stopDataPush` (int id=1)
关闭机器人上的数据推送流
- `Response startControl` (int duration, int startBuffer=5, int filterTypeint=1, int id=1)
以指定参数开启机器人的实时控制
- `Response stopControl` (int id=1)
关闭机器人上的实时控制
- `Response setCollisionSensitivity` (int level, int id=1)
设置机器人上的碰撞灵敏度等级
- `Response setPayload` (int payloadId, int id=1)
设置机器人上的负载参数

静态 Public 成员函数

- static void `printResponse` (const `Codroid::Response` &resp)
打印指令的响应

5.1.1 成员函数说明

5.1.1.1 calculateRelativePose()

```
std::vector<double> Codroid::CodroidControlInterface::calculateRelativePose (
    const RelativePoseParams & params,
    int id = 1 )
```

计算机器人的相对姿态

参数

params	相对姿态参数。
id	请求 ID。

返回

笛卡尔坐标偏移向量。

5.1.1.2 clearError()

```
Response Codroid::CodroidControlInterface::clearError (
    int id = 1 )
```

清除机器人上的错误

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.3 clearStartLine()

```
Response Codroid::CodroidControlInterface::clearStartLine (
    int id = 1 )
```

清除机器人上工程执行的启动行设置

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.4 connect()

```
bool Codroid::CodroidControlInterface::connect (
    const std::string & ip,
    int port = 9001 )
```

连接机器人

参数

ip	机器人IP 地址。
port	TCP 端口号。(默认 9001)

返回

连接成功返回 true。

5.1.1.5 enterRemoteScriptMode()

```
Response Codroid::CodroidControlInterface::enterRemoteScriptMode (
    int id = 1 )
```

在机器人上进入远程脚本模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.6 forwardKinematics()

```
std::vector<double> Codroid::CodroidControlInterface::forwardKinematics (
    const FKParams & params,
    int id = 1 )
```

计算机机器人的正运动学

参数

params	正运动学参数。
id	请求 ID。

返回

关节角度向量。

5.1.1.7 getAI()

```
double Codroid::CodroidControlInterface::getAI (
    int port,
    int id = 1 )
```

获取模拟输入端口的值

参数

port	端口号。
id	请求 ID。

返回

指定模拟输入端口的值。

5.1.1.8 getAO()

```
double Codroid::CodroidControlInterface::getAO (
    int port,
    int id = 1 )
```

获取模拟输出端口的值

参数

port	端口号。
id	请求 ID。

返回

指定模拟输出端口的值。

5.1.1.9 getDI()

```
int Codroid::CodroidControlInterface::getDI (
    int port,
    int id = 1 )
```

获取数字输入端口的值

参数

port	端口号。
id	请求 ID。

返回

指定数字输入端口的值。

5.1.1.10 getDO()

```
int Codroid::CodroidControlInterface::getDO (
    int port,
    int id = 1 )
```


获取数字输出端口的值

参数

port	端口号。
id	请求 ID。

返回

指定数字输出端口的值。

5.1.1.11 getGlobalVars()

```
Response Codroid::CodroidControlInterface::getGlobalVars (
    int id = 1 )
```

获取机器人上的全局变量列表

参数

id	请求 ID。
----	--------

返回

包含全局变量列表的响应对象。

5.1.1.12 getIOValues()

```
std::vector<IOInfo> Codroid::CodroidControlInterface::getIOValues (
    const std::vector< IOInfo > & queryList,
    int id = 1 )
```

获取机器人上IO 端口的状态

参数

queryList	要查询的IO 端口列表。
id	请求 ID。

返回

包含查询IO 端口状态的IOInfo 对象向量。

5.1.1.13 getProjectVar()

```
Response Codroid::CodroidControlInterface::getProjectVar (
    int id = 1 )
```

从机器人上获取工程变量的值

参数

id	请求 ID。
----	--------

返回

包含工程变量值的响应对象。

5.1.1.14 getRegisterValue()

```
double Codroid::CodroidControlInterface::getRegisterValue (
    const int address,
    int id = 1 )
```

获取机器人上单个寄存器的值

参数

address	要查询的寄存器地址。
id	请求 ID。

返回

指定寄存器的值。

5.1.1.15 getRegisterValues()

```
std::vector<RegisterInfo> Codroid::CodroidControlInterface::getRegisterValues (
    const std::vector< int > & addresses,
    int id = 1 )
```

获取机器人上寄存器的值

参数

addresses	要查询的寄存器地址向量。
id	请求 ID。

返回

包含查询寄存器值的RegisterInfo 对象向量。

5.1.1.16 inverseKinematics()

```
std::vector<double> Codroid::CodroidControlInterface::inverseKinematics (
    const IKParams & params,
    int id = 1 )
```

计算机器人的逆运动学

参数

params	逆运动学参数。
id	请求 ID。

返回

关节角度向量。

5.1.1.17 jog()

```
Response Codroid::CodroidControlInterface::jog (
    const JogParams & params,
    int id = 1 )
```

对机器人执行点动操作

参数

params	点动参数。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.18 jogHeartbeat()

```
Response Codroid::CodroidControlInterface::jogHeartbeat (
    int id = 1 )
```

发送点动操作的心跳信号, 至少每 500ms 发送一次

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.19 movC()

```
Response Codroid::CodroidControlInterface::movC (
    const MoveInstruction & inst,
    int id = 1 )
```

按圆弧路径移动机器人

参数

inst	移动指令。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.20 movCircle()

```
Response Codroid::CodroidControlInterface::movCircle (
    const MoveInstruction & inst,
    int id = 1 )
```

按圆弧路径移动机器人，形成一个完整的圆

参数

inst	移动指令。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.21 move()

```
Response Codroid::CodroidControlInterface::move (
    const std::vector< MoveInstruction > & path,
    int id = 1 )
```

按指定路径移动机器人

参数

path	路径。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.22 moveTo()

```
Response Codroid::CodroidControlInterface::moveTo (
    const MoveToParams & params,
    int id = 1 )
```

将机器人移动到特定位置

参数

params	移动到参数。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.23 moveToHeartbeat()

```
Response Codroid::CodroidControlInterface::moveToHeartbeat (
    int id = 1 )
```

发送 moveTo 操作的心跳信号, 至少每 500ms 发送一次

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.24 movJ() [1/2]

```
Response Codroid::CodroidControlInterface::movJ (
    const MoveInstruction & inst,
    int id = 1 )
```

将机器人移动到特定关节位置

参数

inst	移动指令。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.25 movJ() [2/2]

```
Response Codroid::CodroidControlInterface::movJ (
    const std::vector< double > & jp,
    double speed,
    double acc,
    int id = 1 )
```

将机器人移动到特定关节位置

参数

jp	关节位置。
speed	速度
acc	加速度
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.26 movL() [1/2]

```
Response Codroid::CodroidControlInterface::movL (
    const MoveInstruction & inst,
    int id = 1 )
```

按直线路径移动机器人

参数

inst	移动指令。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.27 movL() [2/2]

```
Response Codroid::CodroidControlInterface::movL (
    const std::vector< double > & cp,
    double speed,
    double acc,
    const std::vector< double > & coor = {},
    const std::vector< double > & tool = {},
    int id = 1 )
```

按直线路径移动机器人

参数

cp	笛卡尔位置。
speed	速度
acc	加速度
coor	目标点的可选坐标系，默认是世界坐标系。
tool	目标点的可选工具坐标系。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.28 pauseMove()

```
Response Codroid::CodroidControlInterface::pauseMove (
    int id = 1 )
```


暂停机器人当前的运动

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.29 pauseProject()

```
Response Codroid::CodroidControlInterface::pauseProject (
    int id = 1 )
```

暂停机器人上正在运行的工程

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.30 printResponse()

```
static void Codroid::CodroidControlInterface::printResponse (
    const Codroid::Response & resp ) [static]
```

打印指令的响应

参数

action	操作名称。
resp	响应对象。

5.1.1.31 removeExtendArray()

```
Response Codroid::CodroidControlInterface::removeExtendArray (
    int index,
    int id = 1 )
```

从机器人上删除一个扩展数组索引

参数

index	要删除的扩展数组索引。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.32 removeGlobalVars()

```
Response Codroid::CodroidControlInterface::removeGlobalVars (
    const std::vector< std::string > & varNames,
    int id = 1 )
```

从机器人上删除全局变量

参数

varNames	要删除的变量名称向量。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.33 resumeMove()

```
Response Codroid::CodroidControlInterface::resumeMove (
    int id = 1 )
```

恢复机器人暂停的运动

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.34 resumeProject()

```
Response Codroid::CodroidControlInterface::resumeProject (
    int id = 1 )
```

恢复机器人上暂停的工程

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.35 RS485flush()

```
Response Codroid::CodroidControlInterface::RS485flush (
    int id = 1 )
```

清空机器人上的RS485 通信缓冲区

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.36 RS485init()

```
Response Codroid::CodroidControlInterface::RS485init (
    int baudrate,
    RS485StopBits stopBit = RS485StopBits::One,
    int dataBit = 8,
    RS485Parity parity = RS485Parity::None,
    int id = 1 )
```

初始化机器人上的RS485 通信

参数

baudrate	RS485 通信的波特率。
stopBit	RS485 通信的停止位。
dataBit	RS485 通信的数据位。
parity	RS485 通信的校验位。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.37 RS485read()

```
Response Codroid::CodroidControlInterface::RS485read (
    int length,
    int timeout = 5000,
    int id = 1 )
```

从机器人上的RS485 通信读取数据

参数

length	要读取的字节数。
timeout	读取操作的超时时间。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.38 RS485write()

```
Response Codroid::CodroidControlInterface::RS485write (
    const std::vector< uint8_t > & data,
    int id = 1 )
```

向机器人上的RS485 通信写入数据

参数

data	要写入的数据。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.39 runProject()

```
Response Codroid::CodroidControlInterface::runProject (
    const std::string & projectid,
    int id = 1 )
```

在机器人上通过工程ID 运行工程

参数

projectid	工程 ID。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.40 runProjectByIndex()

```
Response Codroid::CodroidControlInterface::runProjectByIndex (
    int index,
    int id = 1 )
```

在机器人上通过索引运行工程

参数

index	工程索引。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.41 runScript()

```
Response Codroid::CodroidControlInterface::runScript (
    const std::string & script,
    int id = 1 )
```

在机器人上运行脚本

参数

script	脚本内容。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.42 runStep()

```
Response Codroid::CodroidControlInterface::runStep (
    const std::string & projectid,
    int id = 1 )
```

在机器人上单步运行工程

参数

projectid	工程 ID。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.43 saveGlobalVars()

```
Response Codroid::CodroidControlInterface::saveGlobalVars (
    const std::map< std::string, Variable > & vars,
    int id = 1 )
```

将全局变量列表保存到机器人

参数

vars	要保存的全局变量映射。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.44 sendCommand()

```
Response Codroid::CodroidControlInterface::sendCommand (
    const std::string & type,
    const json & data,
    int id )
```

向机器人发送指令

参数

type	指令类型，例如"runScript"、"switchOn" 等。
data	指令数据，JSON 格式，根据不同指令类型具体定义。
id	请求 ID，用于匹配响应，默认为 1。

返回

包含指令执行结果的响应对象。

5.1.1.45 setAO()

```
Response Codroid::CodroidControlInterface::setAO (
    int port,
    double value,
    int id = 1 )
```

设置模拟输出端口的值

参数

port	端口号。
value	要设置的值。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.46 setAutoSpeedRate()

```
Response Codroid::CodroidControlInterface::setAutoSpeedRate (
    int speed,
    int id = 1 )
```

设置机器人的自动运动倍率

参数

speed	速度倍率，百分比形式（0-100）。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.47 setCollisionSensitivity()

```
Response Codroid::CodroidControlInterface::setCollisionSensitivity (
    int level,
    int id = 1 )
```

设置机器人上的碰撞灵敏度等级

参数

level	碰撞灵敏度等级（0-5）。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.48 setDO()

```
Response Codroid::CodroidControlInterface::setDO (
    int port,
    int value,
    int id = 1 )
```

设置数字输出端口的值

参数

port	端口号。
value	要设置的值。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.49 setExtendArrayType()

```
Response Codroid::CodroidControlInterface::setExtendArrayType (
    int index,
    ExtendArrayType type,
    int id = 1 )
```

设置机器人上扩展数组的数据类型

参数

index	扩展数组的索引。
type	要设置的数据类型。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.50 setIOValue()

```
Response Codroid::CodroidControlInterface::setIOValue (
    const std::string & type,
    int port,
    double value,
    int id = 1 )
```

设置机器人上IO 端口的值

参数

type	IO 类型 ("DI"、"DO"、"AI"、"AO")。
port	端口号。
value	要设置的值。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.51 setManualSpeedRate()

```
Response Codroid::CodroidControlInterface::setManualSpeedRate (
    int speed,
    int id = 1 )
```

设置机器人的手动运动倍率

参数

speed	速度倍率，百分比形式（0-100）。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.52 setPayload()

```
Response Codroid::CodroidControlInterface::setPayload (
    int payloadId,
    int id = 1 )
```

设置机器人上的负载参数

参数

payloadId	要设置的负载ID。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.53 setRegisterValue()

```
Response Codroid::CodroidControlInterface::setRegisterValue (
    int address,
    double value,
    int id = 1 )
```

设置机器人上寄存器的值

参数

address	要设置的寄存器地址。
value	要设置的值。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.54 setStartLine()

```
Response Codroid::CodroidControlInterface::setStartLine (
    int startline,
    int id = 1 )
```

设置机器人上工程执行的启动行

参数

startline	要开始执行的行号。
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.55 startControl()

```
Response Codroid::CodroidControlInterface::startControl (
    int duration,
    int startBuffer = 5,
    int filterTypeint = 1,
    int id = 1 )
```

以指定参数开启机器人的实时控制

参数

duration	实时控制的间隔时间，单位为毫秒。（1000/频率）
startBuffer	开始控制前的初始缓冲数组，及收到多少个位置消息后才开始运动
filterTypeint	滤波类型（0-关闭滤波 1-平均滤波值）
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.56 startDataPush()

```
Response Codroid::CodroidControlInterface::startDataPush (
    const std::string & ip,
    int port,
    int duration,
    int id = 1 )
```

开启从机器人到指定IP 和端口的数据推送流

参数

ip	要推送数据的IP 地址。
port	要推送数据的端口号。
duration	数据推送流的间隔时间，单位为毫秒 (1000/频率)
id	请求 ID。

返回

包含指令执行结果的响应对象。

5.1.1.57 startDrag()

```
Response Codroid::CodroidControlInterface::startDrag (
    int id = 1 )
```

进入机器人的拖拽模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.58 stopControl()

```
Response Codroid::CodroidControlInterface::stopControl (
    int id = 1 )
```

关闭机器人上的实时控制

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.59 stopDataPush()

```
Response Codroid::CodroidControlInterface::stopDataPush (
    int id = 1 )
```

关闭机器人上的数据推送流

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.60 stopDrag()

```
Response Codroid::CodroidControlInterface::stopDrag (
    int id = 1 )
```

退出机器人的拖拽模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.61 stopJog()

```
Response Codroid::CodroidControlInterface::stopJog (
    int id = 1 )
```

停止机器人上的点动操作

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.62 stopMove()

```
Response Codroid::CodroidControlInterface::stopMove (
    int id = 1 )
```

立即停止机器人当前的运动

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.63 stopProject()

```
Response Codroid::CodroidControlInterface::stopProject (
    int id = 1 )
```

停止机器人上正在运行的工程

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.64 switchOff()

```
Response Codroid::CodroidControlInterface::switchOff (
    int id = 1 )
```

下使能机器人

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.65 switchOn()

```
Response Codroid::CodroidControlInterface::switchOn (
    int id = 1 )
```

上使能机器人

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.66 toActual()

```
Response Codroid::CodroidControlInterface::toActual (
    int id = 1 )
```

进入机器人的实机模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.67 toAuto()

```
Response Codroid::CodroidControlInterface::toAuto (
    int id = 1 )
```

进入机器人的自动模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.68 toManual()

```
Response Codroid::CodroidControlInterface::toManual (
    int id = 1 )
```

进入机器人的手动模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.69 toRemote()

```
Response Codroid::CodroidControlInterface::toRemote (
    int id = 1 )
```

进入机器人的远程模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

5.1.1.70 toSimulation()

```
Response Codroid::CodroidControlInterface::toSimulation (
    int id = 1 )
```

进入机器人的仿真模式

参数

id	请求 ID。
----	--------

返回

包含指令执行结果的响应对象。

该类的文档由以下文件生成:

- include/Codroid/[CodroidControlInterface.h](#)

5.2 Codroid::CodroidException 类参考

Codroid SDK 专用异常类

```
#include <CodroidDefine.h>
```

Public 成员函数

- CodroidException (const std::string &message)

5.2.1 详细描述

Codroid SDK 专用异常类

该类的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.3 Codroid::CodroidSubscriber 类参考

用于通过独立 TCP 通道进行异步数据订阅的专用类。

```
#include <CodroidSubscriber.h>
```

Public 成员函数

- [CodroidSubscriber](#) ()
构造函数
- [~CodroidSubscriber](#) ()
析构函数
- bool [connect](#) (const std::string &ip, int port=9001)
连接到机器人订阅服务器
- void [disconnect](#) ()
终止连接并停止后台处理
- bool [subscribe](#) (const std::string &topic, int cycle=0)
订阅特定的数据主题
- void [setStatusCallback](#) ([StatusCallback](#) cb)
设置机器人状态更新回调
- void [setPostureCallback](#) ([Codroid::PostureCallback](#) cb)
设置机器人位姿更新回调
- void [setVarCallback](#) ([VarCallback](#) cb)
设置变量更新回调
- void [setLogCallback](#) ([LogCallback](#) cb)
设置系统日志回调

5.3.1 详细描述

用于通过独立 TCP 通道进行异步数据订阅的专用类。

该类管理独立的后台线程，用于处理来自机器人的持续数据推送（如状态、位姿、日志），不会阻塞指令执行。

5.3.2 成员函数说明

5.3.2.1 connect()

```
bool Codroid::CodroidSubscriber::connect (
    const std::string & ip,
    int port = 9001 )
```

连接到机器人订阅服务器

参数

ip	机器人 IP 地址
port	TCP 端口号 (默认 9001)

返回

连接成功返回 true, 否则返回 false

5.3.2.2 setLogCallback()

```
void Codroid::CodroidSubscriber::setLogCallback (
    LogCallback cb ) [inline]
```

设置系统日志回调

参数

cb	回调函数
----	------

5.3.2.3 setPostureCallback()

```
void Codroid::CodroidSubscriber::setPostureCallback (
    Codroid::PostureCallback cb ) [inline]
```

设置机器人位姿更新回调

参数

cb	回调函数
----	------

5.3.2.4 setStatusCallback()

```
void Codroid::CodroidSubscriber::setStatusCallback (
    StatusCallback cb ) [inline]
```

设置机器人状态更新回调

参数

cb	回调函数
----	------

5.3.2.5 setVarCallback()

```
void Codroid::CodroidSubscriber::setVarCallback (
    VarCallback cb ) [inline]
```

设置变量更新回调

参数

cb	回调函数
----	------

5.3.2.6 subscribe()

```
bool Codroid::CodroidSubscriber::subscribe (
    const std::string & topic,
    int cycle = 0 )
```

订阅特定的数据主题

向机器人发送协议 15.1 指令，开始推送指定主题的数据。

参数

topic	主题名称 (例如"RobotStatus", "RobotPosture", "Log")
cycle	推送周期, 单位毫秒 (0 表示默认或仅在变化时推送)

返回

订阅指令发送成功返回 true

该类的文档由以下文件生成:

- include/Codroid/[CodroidSubscriber.h](#)

5.4 Codroid::FKParams 结构体参考

正解参数结构体

```
#include <CodroidDefine.h>
```

Public 成员函数

- FKParams (const std::vector< double > &jointPos)

Public 属性

- `std::vector< double > jp`
[必填] 关节角 [j1...j6], 单位: deg
- `std::vector< double > coor`
[可选] 用户坐标系, 不传则不处理
- `std::vector< double > tool`
[可选] 工具坐标系, 不传则不处理
- `std::vector< double > ep`
[可选] 附加轴位置

5.4.1 详细描述

正解参数结构体

该结构体的文档由以下文件生成:

- `include/Codroid/CodroidDefine.h`

5.5 Codroid::IKParams 结构体参考

逆解参数结构体

```
#include <CodroidDefine.h>
```

Public 成员函数

- `IKParams (const std::vector< double > &cartesianPos)`

Public 属性

- `std::vector< double > cp`
[必填] 笛卡尔末端位置, 单位: mm, deg
- `std::vector< double > rj`
[可选] 参考关节角, 默认 [20,20,20,20,20,20]
- `std::vector< double > ep`
[可选] 附加轴位置

5.5.1 详细描述

逆解参数结构体

该结构体的文档由以下文件生成:

- `include/Codroid/CodroidDefine.h`

5.6 Codroid::IOInfo 结构体参考

IO 信息结构体

```
#include <CodroidDefine.h>
```

Public 成员函数

- IOInfo (const std::string &t, int p)

Public 属性

- std::string [type](#)
IO 类型
- int [port](#)
端口号
- double [value](#)
IO 数值

5.6.1 详细描述

IO 信息结构体

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.7 Codroid::JogParams 结构体参考

点动参数结构体

```
#include <CodroidDefine.h>
```

Public 成员函数

- JogParams (JogMode m, double s, int i, [CoordType](#) ct=CoordType::User, int cid=1)

Public 属性

- JogMode [mode](#) = JogMode::Line
1: 关节点动 2: 直线点动
- double [speed](#) = 0.0
速度范围 -1~1
- int [index](#) = 1
轴/关节序号
- [CoordType](#) [coordType](#) = CoordType::User
用户系工具系
- int [coordId](#) = 1
坐标系 ID

5.7.1 详细描述

点动参数结构体

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.8 Codroid::MoveInstruction 结构体参考

单条运动指令详情

```
#include <CodroidDefine.h>
```

Public 属性

- MoveType [type](#) = MoveType::movJ
运动类型
- double [speed](#) = 60.0
运动速度
- double [acc](#) = 150.0
加速度
- double [blend](#) = -1.0
过渡半径
- double [relativeBlend](#) = -1.0
相对过渡百分比
- int [circleNum](#) = 1
圆周运动圈数
- MovePoint [targetPoint](#)
目标点
- MovePoint [middlePoint](#)
中间点
- std::vector< double > [coord](#)
用户坐标系
- std::vector< double > [tool](#)
工具坐标系

5.8.1 详细描述

单条运动指令详情

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.9 Codroid::MoveToParams 结构体参考

运动参数

```
#include <CodroidDefine.h>
```

Public 成员函数

- MoveToParams (MoveToType t)
- MoveToParams (MoveToType t, const [MoveToTarget](#) &tgt)

Public 属性

- MoveToType type = MoveToType::Home
- [MoveToTarget](#) target
[可选] 目标位置

5.9.1 详细描述

运动参数

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.10 Codroid::MoveToTarget 结构体参考

运动目标位置

```
#include <CodroidDefine.h>
```

Public 成员函数

- [MoveToTarget](#) ()=default
[可选] 关节位置 [j1..j6]

静态 Public 成员函数

- static [MoveToTarget](#) Joint (const std::vector< double > &j)
- static [MoveToTarget](#) Cartesian (const std::vector< double > &c)

Public 属性

- std::vector< double > [cp](#)
[可选] 末端位置 [x,y,z,a,b,c]
- std::vector< double > [jp](#)

5.10.1 详细描述

运动目标位置

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.11 Codroid::ProjectState 结构体参考

工程执行状态

```
#include <CodroidDefine.h>
```

Public 属性

- std::string [id](#)
工程 ID
- int [state](#)
工程状态
- bool [isStep](#)
是否单步运行
- std::map< std::string, int > [scriptLines](#)
脚本ID 对应行号

5.11.1 详细描述

工程执行状态

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.12 Codroid::RegisterInfo 结构体参考

寄存器信息结构体

```
#include <CodroidDefine.h>
```

Public 成员函数

- RegisterInfo (int addr, double val)

Public 属性

- int [address](#)
寄存器地址
- double [value](#)
寄存器数值

5.12.1 详细描述

寄存器信息结构体

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.13 Codroid::RelativePoseParams 结构体参考

相对位姿计算参数

```
#include <CodroidDefine.h>
```

Public 成员函数

- RelativePoseParams (const std::vector< double > &p, const std::vector< double > &o, [CoordType](#) type)

Public 属性

- std::vector< double > [pos](#)
当前末端TCP 坐标 [x,y,z,a,b,c]
- std::vector< double > [offset](#)
偏移量 [x,y,z,a,b,c]
- [CoordType](#) [coordType](#) = CoordType::Tool
[可选] 坐标系类型
- std::vector< double > [posCoord](#)
[可选] 当前末端TCP 坐标系, 默认世界坐标系
- std::vector< double > [coord](#)
[可选] [coordType](#) 为 user 时有效, 偏移坐标系, 默认世界坐标系

5.13.1 详细描述

相对位姿计算参数

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.14 Codroid::Response 结构体参考

SDK 标准响应结构体

```
#include <CodroidDefine.h>
```

Public 属性

- int [id](#)
请求 ID
- std::string [ty](#)
请求类型
- json [db](#)
返回数据内容
- std::string [error_msg](#)
错误信息（成功则为空）

5.14.1 详细描述

SDK 标准响应结构体

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.15 Codroid::RobotPosture 结构体参考

机器人实时位姿

```
#include <CodroidDefine.h>
```

Public 属性

- std::vector< double > [joint](#)
关节角 (度)
- std::vector< double > [cart](#)
笛卡尔坐标 [x,y,z,a,b,c]

5.15.1 详细描述

机器人实时位姿

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.16 Codroid::RobotStatus 结构体参考

机器人运行状态

```
#include <CodroidDefine.h>
```

Public 属性

- int [mode](#)
0: 手动, 1: 自动, 2: 远程
- int [state](#)
0: 未使能, 1: 使能中, 2: 空闲, 3: 点动中, 4: RunTo, 5: 拖动中
- int [isMoving](#)
0: 停止, 1: 运动
- double [moveRate](#)
自动速度倍率
- double [manualMoveRate](#)
手动速度倍率
- int [recoveryState](#)
- bool [isSimulation](#)
- int [teachingPendant](#)
- int [rescueFlag](#)
- int [modeSwitch](#)
- int [ToolId](#)
- int [PayloadId](#)
- int [CoordinateId](#)
- int [defaultToolId](#)
- int [defaultPayloadId](#)
- int [defaultUserCoordId](#)
- std::string [type](#)
机器人型号
- std::string [stateName](#)
状态名称
- long long [timestamp](#)
推送时间戳

5.16.1 详细描述

机器人运行状态

该结构体的文档由以下文件生成:

- include/Codroid/[CodroidDefine.h](#)

5.17 Codroid::Variable 结构体参考

全局变量信息结构

```
#include <CodroidDefine.h>
```

Public 成员函数

- `template<typename T >`
`Variable (const T &value, const std::string ¬e="")`

Public 属性

- `std::string val`
变量值 (JSON 字符串格式)
- `std::string nm`
变量备注

5.17.1 详细描述

全局变量信息结构

该结构体的文档由以下文件生成:

- `include/Codroid/CodroidDefine.h`

Chapter 6

文件说明

6.1 include/Codroid/CodroidControllInterface.h 文件参考

控制接口头文件

```
#include "CodroidDefine.h"
#include <unordered_set>
#include <asio.hpp>
#include <memory>
#include <string>
```

类

- class [Codroid::CodroidControllInterface](#)

6.1.1 详细描述

控制接口头文件

6.2 include/Codroid/CodroidDefine.h 文件参考

定义文件

```
#include <string>
#include <vector>
#include <map>
#include <nlohmann/json.hpp>
#include "CodroidExport.h"
#include <stdexcept>
#include <functional>
```

类

- struct [Codroid::Response](#)
SDK 标准响应结构体
- class [Codroid::CodroidException](#)
Codroid SDK 专用异常类
- struct [Codroid::MoveInstruction](#)
单条运动指令详情
- struct [Codroid::JogParams](#)
点动参数结构体
- struct [Codroid::RelativePoseParams](#)
相对位姿计算参数
- struct [Codroid::Variable](#)
全局变量信息结构
- struct [Codroid::FKParams](#)
正解参数结构体
- struct [Codroid::IKParams](#)
逆解参数结构体
- struct [Codroid::MoveToTarget](#)
运动目标位置
- struct [Codroid::MoveToParams](#)
运动参数
- struct [Codroid::RobotStatus](#)
机器人运行状态
- struct [Codroid::RobotPosture](#)
机器人实时位姿
- struct [Codroid::ProjectState](#)
工程执行状态
- struct [Codroid::IOInfo](#)
IO 信息结构体
- struct [Codroid::RegisterInfo](#)
寄存器信息结构体

类型定义

- using [Codroid::json](#) = nlohmann::json
- using [Codroid::TopicCallback](#) = std::function< void(const std::string &, const nlohmann::json &)>

枚举

- enum class [Codroid::RS485Parity](#) : int { None = 0 , Odd = 1 , Even = 2 }
 - enum class [Codroid::RS485StopBits](#) : int { One = 1 , Two = 2 }
 - enum class [Codroid::CoorType](#) { Tool , User }
- 坐标系类型

函数

- `Codroid::NLOHMANN_JSON_SERIALIZE_ENUM` (CoorType, { {CoorType::Tool, "tool"}, {CoorType::User, "user"} }) enum class MoveType
运动插补类型
- `NLOHMANN_JSON_SERIALIZE_ENUM`(MoveType, { {MoveType::movJ, "movJ"}, {MoveType::movL, "movL"}, {MoveType::movC, "movC"}, {MoveType::movCircle, "movCircle"} }) enum class MoveToType `Codroid::NLOHMANN_JSON_SERIALIZE_ENUM` (MoveToType, { {MoveToType::Home, 0}, {MoveToType::Safe, 1}, {MoveToType::Candle, 2}, {MoveToType::Packing, 3}, {MoveToType::Joint, 4}, {MoveToType::Line, 5}, {MoveToType::ResumePoint, 6} }) enum class ExtendArrayType
运动类型
- `Codroid::NLOHMANN_JSON_SERIALIZE_ENUM` (ExtendArrayType, { {ExtendArrayType::Bool, "Bool"}, {ExtendArrayType::UInt8, "UInt8"}, {ExtendArrayType::Int8, "Int8"}, {ExtendArrayType::UInt16, "UInt16"}, {ExtendArrayType::Int16, "Int16"}, {ExtendArrayType::UInt32, "UInt32"}, {ExtendArrayType::Int32, "Int32"}, {ExtendArrayType::Float32, "Float32"} }) enum class JogMode
- `Codroid::NLOHMANN_JSON_SERIALIZE_ENUM` (JogMode, { {JogMode::Joint, 1}, {JogMode::Line, 2} }) struct MovePoint
运动位姿点定义

6.2.1 详细描述

定义文件

6.2.2 函数说明

6.2.2.1 NLOHMANN_JSON_SERIALIZE_ENUM() [1/2]

```
Codroid::NLOHMANN_JSON_SERIALIZE_ENUM (
    JogMode ,
    { {JogMode::Joint, 1}, {JogMode::Line, 2} } )
```

运动位姿点定义

< 关节角 (单位: 度)

< 笛卡尔坐标 (单位:mm, 度)

< 逆解参考关节角

< 附加轴位置

6.2.2.2 NLOHMANN_JSON_SERIALIZE_ENUM() [2/2]

```
NLOHMANN_JSON_SERIALIZE_ENUM (MoveType, { {MoveType::movJ, "movJ"}, {MoveType::movL, "movL"}, {MoveType::movC, "movC"}, {MoveType::movCircle, "movCircle"} }) enum class MoveToType Codroid::NLOHMANN_JSON_SERIALIZE_ENUM (
    MoveToType ,
    { {MoveToType::Home, 0}, {MoveToType::Safe, 1}, {MoveToType::Candle, 2}, {MoveToType::Packing, 3}, {MoveToType::Joint, 4}, {MoveToType::Line, 5}, {MoveToType::ResumePoint, 6} } )
```

运动类型

扩展数组支持的数据类型

6.3 include/Codroid/CodroidSubscriber.h 文件参考

CodroidSubscriber 类的头文件

```
#include <asio.hpp>
#include <nlohmann/json.hpp>
#include <string>
#include <thread>
#include <atomic>
#include <functional>
#include <vector>
#include <map>
#include <mutex>
#include "CodroidDefine.h"
```

类

- class [Codroid::CodroidSubscriber](#)
用于通过独立 TCP 通道进行异步数据订阅的专用类。

类型定义

- using [Codroid::StatusCallback](#) = std::function< void(const RobotStatus &)>
机器人状态回调类型
- using [Codroid::PostureCallback](#) = std::function< void(const RobotPosture &)>
机器人位姿回调类型
- using [Codroid::VarCallback](#) = std::function< void(const std::map< std::string, std::string > &)>
变量更新回调类型
- using [Codroid::LogCallback](#) = std::function< void(const std::string &msg, int level)>
系统日志回调类型

6.3.1 详细描述

CodroidSubscriber 类的头文件